

Task 35: Task Scheduler

Problem Description

Given a characters array `tasks`, representing the tasks a CPU needs to do, where each letter represents a different task. Tasks could be done in any order. Each task is done in one unit of time. For each unit of time, the CPU could complete either one task or just be idle. However, there is a non-negative integer `n` that represents the cooldown period between two same tasks. Return the least number of units of times that the CPU will take to finish all the given tasks.

Java Solution

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] freq = new int[26];
        int maxFreq = 0;
        int maxCount = 0;

        for (char task : tasks) {
            freq[task - 'A']++;
            if (freq[task - 'A'] > maxFreq) {
                maxFreq = freq[task - 'A'];
                maxCount = 1;
            } else if (freq[task - 'A'] == maxFreq) {
                maxCount++;
            }
        }

        int partCount = maxFreq - 1;
        int partLength = n - (maxCount - 1);
        int emptySlots = partCount * partLength;
        int availableTasks = tasks.length - maxFreq * maxCount;
        int idles = Math.max(0, emptySlots - availableTasks);

        return tasks.length + idles;
    }
}
```